



PRÁCTICA FINAL

TechStore – Tienda online con Web Components

1. CONTEXTO

Una empresa de periféricos informáticos desea desarrollar una **mini tienda online moderna**.

El objetivo es construir una aplicación web utilizando:

- **Web Components**
- **Arquitectura por capas**
- **Gestión de estado global**

2. OBJETIVOS

Con esta práctica aprenderás a:

- Crear **Custom Elements** reutilizables
- Utilizar **Shadow DOM**
- Organizar código en **capas (components, models, services, state)**
- Implementar un **estado global (patrón observer)**
- Reutilizar código mediante **componentes base**
- Gestionar eventos sin acoplar componentes

3. ESTRUCTURA DEL PROYECTO

Debes organizar tu proyecto de la siguiente forma:



`</> Bash`

```
.
├── img
│   └── ...
├── index.html
├── js
│   ├── components
│   │   ├── ProductCardComponent.js
│   │   ├── CartComponent.js
│   │   ├── CartStatusComponent.js
│   │   ├── CounterComponent.js
│   │   ├── CategoryFilterComponent.js
│   │   └── ProductListComponent.js
│   ├── libcomponents
│   │   ├── base_component.js
│   │   ├── scaffold_component.js
│   │   └── tab_component.js
│   ├── main.js
│   ├── models
│   │   └── product.js
│   ├── services
│   │   └── productService.js
│   └── state
│       └── Cart.js
```

4. COMPONENTES A DESARROLLAR

4-1 <product-card>

- Mostrar:
 - nombre
 - precio
 - botón “Añadir”
- Añade producto al carrito usando el estado global



4-2 <product-list>

- Recibe productos desde main.js
- Genera dinámicamente los <product-card>

4-3 <cart-component>

- Muestra productos añadidos
- Botón "Vaciar carrito"

4-4 <cart-status>

- Muestra número de productos en el carrito
- Se actualiza automáticamente

4-5 <category-filter>

- Filtra productos por categoría

4-6 <counter-component>

- Muestra el total de productos o precio

5. COMPONENTE BASE (OBLIGATORIO)

Todos los componentes deben heredar de:

```
// libcomponents/base_component.js
export class BaseComponent extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: "open" });
  }
  render(html) {
    this.shadowRoot.innerHTML = html;
  }
}
```



6. MODELO DE DATOS

Debes crear una clase Product en:

```
// models/product.js
export class Product {
  constructor(id, name, category, price, description,
    image) {
    this.id = id;
    this.name = name;
    this.category = category;
    this.price = price;
    this.description = description;
    this.image = image;
  }
}
```

7. SERVICIO DE DATOS

En:

js/services/productService.js

Debes crear una función que devuelva un array de productos.

Ejemplo:

```
// services/productService.js
export function getProducts() {
  return [
    { id: 1, name: "Ratón", category: "raton", price: 25 },
    { id: 2, name: "Teclado", category: "teclado", price:
70 }
  ];
}
```



8. ESTADO GLOBAL (CLAVE DE LA PRÁCTICA)

```
// state/Cart.js
export const Cart = {
  items: [],
  listeners: [],
  subscribe(callback) {
    this.listeners.push(callback);
  },
  notify() {
    this.listeners.forEach(cb => cb(this.items));
  },
  add(product) {
    this.items.push(product);
    this.notify();
  },
  clear() {
    this.items = [];
    this.notify();
  }
};
```

8. SCRIPT PRINCIPAL

En main.js:

- Cargar productos
- Pasarlos al <product-list>
- Gestionar eventos del carrito

9. MAIN.JS

- Cargar productos
- Pasarlos a <product-list>
- Inicializar aplicación



10. INDEX.HTML

Debe incluir:

```
<cart-status></cart-status>

<category-filter></category-filter>

<product-list></product-list>

<cart-component></cart-component>

<script type="module" src="js/main.js"></script>
```

11. FUNCIONAMIENTO OBLIGATORIO

- ✓ Añadir productos al carrito
- ✓ Actualización automática de componentes
- ✓ Uso de estado global
- ✓ Componentes desacoplados
- ✓ Código modular

12. FUNCIONALIDADES EXTRA

- Guardar carrito en localStorage
- Mostrar total del carrito
- Añadir cantidades
- Mejorar diseño con CSS

11. REQUISITOS TÉCNICOS

- Usar **Shadow DOM** en todos los componentes
- Usar customElements.define
- Código organizado en módulos (import/export)
- Nombres de componentes en formato:



nombre-componente

12. REQUISITOS TÉCNICOS

- Uso obligatorio de:
 - Shadow DOM
 - extends HTMLElement
 - customElements.define
 - import / export
- NO usar librerías externas (Bootstrap, React, etc.)

12. CRITERIOS DE EVALUACIÓN

CRITERIO	PUNTOS
WEB COMPONENTS CORRECTOS	2
USO DE SHADOW DOM	2
ESTRUCTURA DEL PROYECTO	2
FUNCIONALIDAD DEL CARRITO	2
CÓDIGO LIMPIO Y ORGANIZADO	1
FUNCIONALIDADES EXTRA	1

13. ENTREGA

- Carpeta comprimida (.zip)
- Debe funcionar al abrir index.html
- Código comentado



TechStore PRO

Carrito: 3 [Ver Carrito](#)

[Todos](#) [Ratones](#) [Teclados](#) [Auriculares](#)



Ratón Gaming

[Ratones](#)

Ratón RGB con sensor óptico.

25 €

[Añadir al Carrito](#)



Teclado Mecánico

[Teclados](#)

Teclado con switches blue.

70 €

[Añadir al Carrito](#)



Auriculares Inalámbricos

[Auriculares](#)

Auriculares Bluetooth con micrófono.

45 €

[Añadir al Carrito](#)



Ratón Inalámbrico

[Ratones](#)

Ratón ergonómico inalámbrico.

20 €

[Añadir al Carrito](#)

Carrito de Compras

- Ratón Gaming x1 - 25 €
- Teclado Mecánico x1 - 70 €
- Auriculares Inalámbricos x1 - 45 €

Total: 140 €

[Vaciar Carrito](#)